

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación a Distancia

Trabajo Práctico Integrador (TPI)

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Materia:

Programación 1

Estudiante:

Mariano Avendaño

Modalidad:

Trabajo individual

Fecha de entrega:

Junio de 2026

Índice

1. Marco Teórico	3
1.1 Listas	3
1.2 Dicionarios	3
1.3 Funciones	3
1.4 Estructuras Condicionales	4
1.5 Ordenamientos	4
1.6 Estadísticas Básicas	5
1.7 Archivos CSV (lectura nativa)	5
2. Decisiones Técnicas y Arquitectura	6
2.1 Estructura de Datos Elegida	6
2.2 Validaciones Implementadas	6
2.3 Diagrama de Flujo	7
2.4 Capturas de Ejecución	8
3. Dificultades y Conclusiones	10
3.1 Dificultades Encontradas	10
3.2 Conclusiones	10
4. Bibliografía y Webgrafía	11
5. Enlaces del Proyecto	11

1. Marco Teórico

A continuación se describen los principales conceptos de Python aplicados en este proyecto, con su justificación y ejemplos tomados directamente del código fuente.

1.1 Listas

Una lista en Python es una estructura de datos ordenada y mutable que permite almacenar múltiples elementos. Se define con corchetes [] y soporta acceso por índice, iteración, inserción y eliminación de elementos en tiempo de ejecución.

En este proyecto, la lista `países[]` es la estructura central que contiene todos los registros cargados desde el CSV. Su naturaleza mutable permite agregar nuevos países (`append()`), recorrerla con `for`, y pasarla por referencia a cada función del sistema sin necesidad de copias:

```
países = [] # lista global vacía al inicio
países.append({'nombre': 'Argentina', 'poblacion': 45376763, ...})
for pais in países: # iteración para mostrar, filtrar u ordenar
    print(pais['nombre'])
```

La lista actúa también como parámetro mutable en funciones como `agregar_pais()` y `actualizar_pais()`, de modo que las modificaciones son visibles desde el menú principal sin necesidad de valores de retorno adicionales.

1.2 Diccionarios

Un diccionario (`dict`) en Python almacena pares clave-valor. El acceso a los datos se realiza por clave en lugar de por índice, lo que hace el código más legible y menos propenso a errores de posición.

Cada país del sistema se modela como un diccionario con cuatro claves fijas:

```
pais = {
    'nombre': 'Brasil',
    'poblacion': 213993437,
    'superficie': 8515767,
    'continente': 'America'
}
```

Este diseño permite acceder de forma natural a cualquier atributo (`pais['poblacion']`) sin depender de índices numéricos, y facilita el ordenamiento y filtrado por cualquier campo usando `lambda`:

```
sorted(países, key=lambda p: p['poblacion'], reverse=True)
```

El diccionario `por_continente` en `mostrar_estadisticas()` utiliza el método `.get()` para contar países por continente sin inicializar manualmente cada clave:

```
por_continente[continente] = por_continente.get(continente, 0) + 1
```

1.3 Funciones

El principio de diseño más importante aplicado en este proyecto es la modularización: una función, una responsabilidad. Esto facilita el mantenimiento, la prueba individual de cada componente y la lectura del código.

El sistema cuenta con 14 funciones independientes organizadas en capas:

- Capa de persistencia: `crear_csv()`, `cargar_paises_desde_csv()`, `guardar_paises_en_csv()`
- Capa de visualización: `mostrar_paises()`
- Capa de operaciones: `agregar_pais()`, `actualizar_pais()`, `buscar_pais()`, `mostrar_estadisticas()`

- Capa de filtros: `filtrar_paises()`, `filtrar_por_continente()`, `filtrar_por_rango_poblacion()`, `filtrar_por_rango_superficie()`
- Capa de ordenamiento: `ordenar_paises()`, `pedir_orden()`
- Utilitarios: `pedir_entero()`
- Punto de entrada: `main()`

La función `pedir_entero()` es un buen ejemplo de función utilitaria que encapsula la validación de enteros en un bucle `while`, evitando repetir el mismo bloque `try/except` en múltiples lugares:

```
def pedir_entero(mensaje):
    while True:
        try:
            return int(input(mensaje).strip())
        except ValueError:
            print('-> Error: Debe ingresar un número entero válido.')
```

1.4 Estructuras Condicionales

Las estructuras `if/elif/else` controlan el flujo del menú principal, seleccionan el tipo de búsqueda (exacta vs. parcial), determinan el criterio de ordenamiento y ejecutan cada subopción del menú de filtros.

Un uso representativo es la bifurcación en `buscar_pais()`, donde el modo ingresado por el usuario determina el tipo de comparación aplicada:

```
if modo == 'E' and nombre == busqueda:
    resultados.append(pais)
elif modo == 'P' and busqueda in nombre:
    resultados.append(pais)
```

Las condiciones de validación también usan estructuras condicionales para rechazar entradas inválidas antes de modificar el estado del sistema:

```
if poblacion <= 0:
    raise ValueError('La población debe ser un entero mayor a 0.')
```

1.5 Ordenamientos

Python ofrece la función incorporada `sorted()` que devuelve una nueva lista ordenada sin modificar la original. Recibe tres argumentos clave: el iterable, `key` (función que extrae el valor de comparación) y `reverse` (`True` para orden descendente).

En `ordenar_paises()`, la clave de ordenamiento se determina según la opción elegida y se usa una expresión `lambda` para extraer el campo correspondiente de cada diccionario:

```
ordenados = sorted(paises, key=lambda p: p[clave], reverse=descendente)
```

El parámetro `clave` puede ser `'nombre'`, `'poblacion'` o `'superficie'`. Cuando `clave` es `'nombre'`, `sorted()` aplica comparación lexicográfica de cadenas; cuando es un campo numérico, aplica comparación aritmética. La función `pedir_orden()` encapsula la interacción con el usuario para determinar el valor de descendente:

```
def pedir_orden():
    while True:
        orden = input('Orden ascendente (A) o descendente (D)? ').strip().upper()
        if orden == 'A': return False
        if orden == 'D': return True
        print('-> Error: Debe ingresar A o D.')
```

La lista original `países[]` nunca se modifica; el resultado ordenado solo se usa para la visualización, garantizando la integridad de los datos en memoria.

1.6 Estadísticas Básicas

El módulo de estadísticas recorre la lista completa en una sola pasada para calcular simultáneamente todos los indicadores, sin necesidad de recorridos adicionales:

- País con mayor y menor población (comparación elemento a elemento)
- Promedio de población (suma total / cantidad de países)
- Promedio de superficie (suma total / cantidad de países)
- Cantidad de países por continente (diccionario de frecuencias)

El recorrido único es eficiente incluso para datasets grandes:

```
mayor, menor = países[0], países[0]
total_poblacion = total_superficie = 0
por_continente = {}

for pais in países:
    total_poblacion += pais['poblacion']
    total_superficie += pais['superficie']
    if pais['poblacion'] > mayor['poblacion']: mayor = pais
    if pais['poblacion'] < menor['poblacion']: menor = pais
    cont = pais['continente']
    por_continente[cont] = por_continente.get(cont, 0) + 1

promedio_pob = total_poblacion / len(países)
promedio_sup = total_superficie / len(países)
```

1.7 Archivos CSV (lectura nativa con `open()`)

Los archivos CSV (Comma-Separated Values) son archivos de texto plano donde cada línea representa un registro y los campos se separan por comas. Son ampliamente utilizados para intercambio de datos por su simplicidad y compatibilidad universal.

En este proyecto, la lectura y escritura del CSV se implementa con la función nativa `open()` de Python, sin importar módulos externos. Esto demuestra comprensión directa de la manipulación de archivos y cadenas de texto:

```
# Lectura
with open(ARCHIVO_CSV, 'r', encoding='utf-8') as archivo:
    lineas = archivo.readlines()

for numero_fila in range(1, len(lineas)): # salta el encabezado
    partes = lineas[numero_fila].strip().split(',')
    nombre = partes[0].strip()
    poblacion = int(partes[1].strip())
    superficie = int(partes[2].strip())
    continente = partes[3].strip()

# Escritura
with open(ARCHIVO_CSV, 'w', encoding='utf-8') as archivo:
    archivo.write('nombre,poblacion,superficie,continente\n')
    for pais in países:
        archivo.write(f"{pais['nombre']},{pais['poblacion']},")
        archivo.write(f"{pais['superficie']},{pais['continente']}\n")
```

Cada fila se valida antes de incorporarla a la lista: se verifica que tenga exactamente 4 campos, que los valores numéricos sean enteros positivos, y en caso de error se emite una advertencia sin detener la carga del resto del archivo.

2. Decisiones Técnicas y Arquitectura

2.1 Estructura de Datos Elegida

Se eligió una lista de diccionarios como estructura de datos principal porque:

- Las listas permiten iterar, filtrar y ordenar con sintaxis nativa de Python.
- Los diccionarios ofrecen acceso por clave legible (`pais['nombre']`) en lugar de índices numéricos.
- La combinación es directamente compatible con los archivos CSV: cada fila del CSV se convierte en un diccionario, y el conjunto de filas en una lista.
- No se requieren importaciones de módulos adicionales para su uso.

La lista vive en memoria RAM durante la ejecución. Cada alta o actualización persiste inmediatamente en `países.csv` para evitar pérdida de datos. La opción 8 del menú permite recargar el archivo si fue editado externamente.

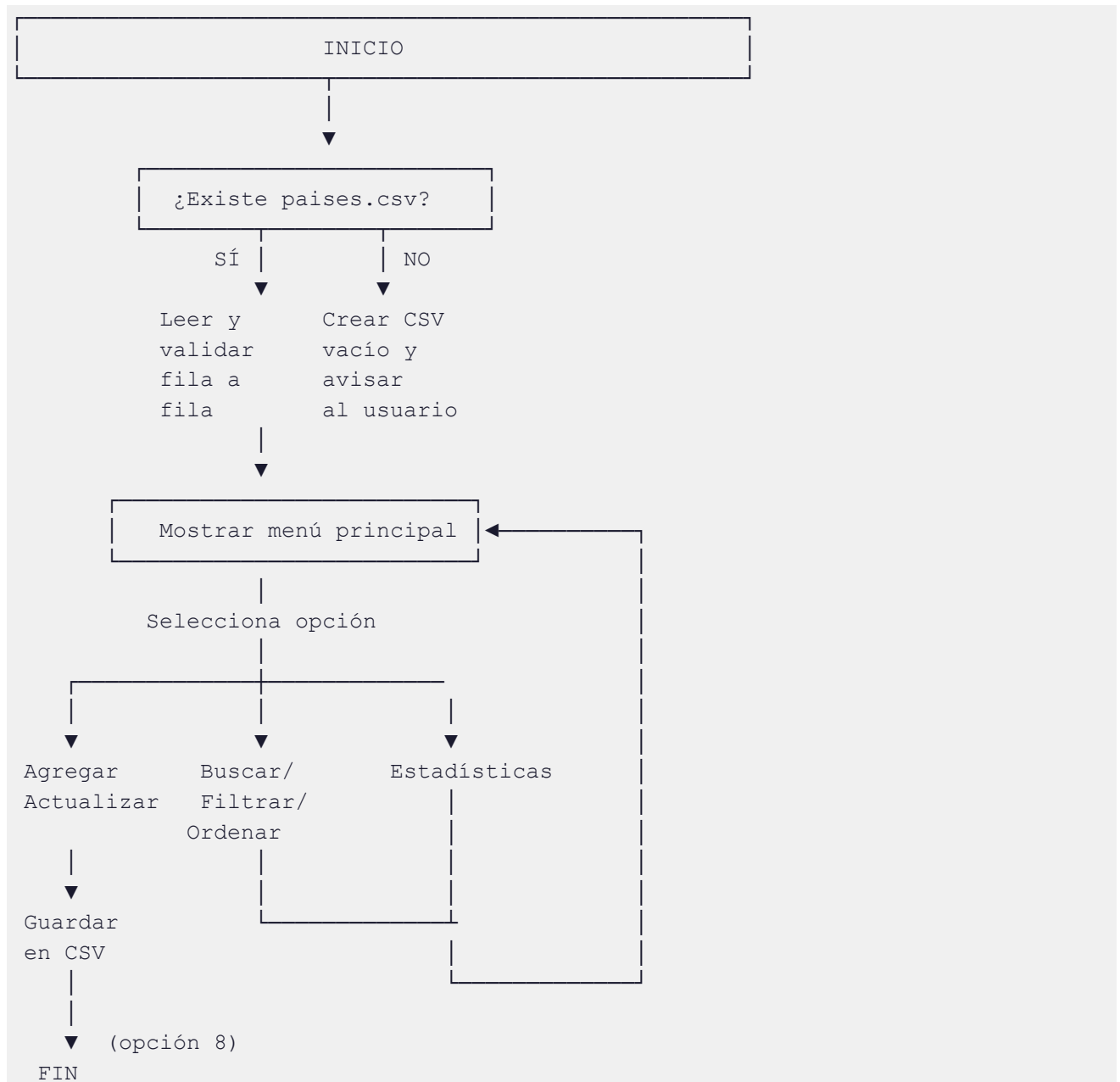
2.2 Validaciones Implementadas

El sistema aplica tres niveles de validación:

Nivel	Validaciones
CSV (carga)	Exactamente 4 campos, valores numéricos convertibles a int, valores > 0. Filas inválidas se omiten con advertencia.
Alta / Actualización	Nombre y continente no vacíos ni números; nombre no duplicado ni números; población y superficie > 0 como enteros. Y también que no sean letras
Filtros y menú	Rango mínimo ≤ máximo; modo de búsqueda E/P; opciones de menú numéricas y dentro de rango.

2.3 Diagrama de Flujo

El siguiente esquema describe el ciclo de vida de la aplicación:



2.4 Capturas de Ejecución

Ejemplo 1 — Inicio del programa y carga del CSV:

```
$ python Avendaño_Mariano_TPI.py
-> Se cargaron 15 países desde 'países.csv'.

=====
GESTIÓN DE DATOS DE PAÍSES
=====
1. Mostrar todos los países
2. Agregar un país
3. Actualizar población y superficie
4. Buscar por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Salir
=====
Seleccione una opción:
```

Ejemplo 2 — Mostrar todos los países (opción 1):

```
--- Listado de países ---
Nombre                Población      Superficie Continente
-----
Argentina             45,376,763     2,780,400 America
Japon                 125,800,000     377,975 Asia
Brasil                213,993,437     8,515,767 America
Alemania              83,149,300      357,022 Europa
...
Total: 15 país(es).
```

Ejemplo 3 — Agregar un país con validación (opción 2):

```
--- Agregar país ---
Ingrese el nombre del país: Argentina
-> Error: 'Argentina' ya se encuentra registrado.
Ingrese el nombre del país: Portugal
Ingrese el continente: Europa
Ingrese la población: -100
-> Error: La población debe ser un numero entero.
Ingrese el nombre del país: Portugal
Ingrese el continente: Europa
Ingrese la población: 10270865
Ingrese la superficie en km2: 92212
-> País 'Portugal' agregado con éxito.
-> Datos guardados correctamente en el CSV.
```

Ejemplo 4 — Búsqueda parcial (opción 4):

```
--- Buscar país ---
Ingrese el nombre o parte del nombre a buscar: bras
¿Búsqueda exacta (E) o parcial (P)? [P]: P
```



```

--- Resultados de búsqueda (parcial) ---
Nombre                Población            Superficie Continente
-----
Brasil                213,993,437          8,515,767 America
Total: 1 país(es).

```

Ejemplo 5 — Filtrar por continente (opción 5):

```

--- Filtros ---
1. Por continente
2. Por rango de población
3. Por rango de superficie
Seleccione una opción: 1
Ingrese el continente a filtrar: Europa

--- Países de Europa ---
Nombre                Población            Superficie Continente
-----
Alemania              83,149,300           357,022 Europa
Francia               67,897,000           551,695 Europa
Noruega               5,421,241            323,802 Europa
Total: 3 país(es).

```

Ejemplo 6 — Ordenar por superficie descendente (opción 6):

```

--- Ordenamiento ---
1. Por nombre
2. Por población
3. Por superficie
Seleccione una opción: 3
Orden ascendente (A) o descendente (D)? D

--- Países ordenados por superficie (descendente) ---
Nombre                Población            Superficie Continente
-----
Canada               38,008,000           9,984,670 America
Brasil               213,993,437           8,515,767 America
Australia            25,687,000           7,692,024 Oceania
India                1,380,004,385         3,287,263 Asia
Argentina            45,376,763           2,780,400 America
...

```

Ejemplo 7 — Estadísticas (opción 7):

```

--- Estadísticas ---
País con mayor población: India (1,380,004,385 hab.)
País con menor población: Uruguay (3,473,730 hab.)
Promedio de población: 168,978,909.53 hab.
Promedio de superficie: 2,716,785.27 km²

Cantidad de países por continente:
- Africa: 3
- America: 6
- Asia: 2
- Europa: 3
- Oceania: 1

```

3. Dificultades y Conclusiones

3.1 Dificultades Encontradas

Durante el desarrollo del proyecto surgieron varios desafíos técnicos que requirieron investigación y decisiones de diseño:

- Validación fila a fila del CSV sin interrumpir la carga: el enfoque inicial fallaba ante cualquier fila inválida, deteniendo la carga completa. La solución fue envolver cada fila en un bloque try/except independiente, registrar una advertencia y continuar con la siguiente fila. Esto permite que un CSV parcialmente corrupto sea útil en lugar de inutilizable.
- Sincronización entre memoria y disco: garantizar que los datos en la lista `países[]` siempre reflejen el estado del CSV (y viceversa) requirió definir una política clara: cualquier alta o actualización llama a `guardar_paises_en_csv()` de forma inmediata.
- Manejo de tipos en la búsqueda y filtros: el módulo comparativo convierte ambos lados a minúsculas antes de comparar (`lower()`), evitando fallos por diferencias de capitalización en el CSV. Los rangos numéricos se validan explícitamente para asegurar que el mínimo no supere al máximo.
- Modularización del submenú de filtros: decidir dónde dividir `filtrar_paises()` requirió evaluar el equilibrio entre cohesión (cada función hace una sola cosa) y acoplamiento (no duplicar la lectura de la lista). La solución fue una función coordinadora que delega a tres funciones especializadas.

3.2 Conclusiones

El desarrollo de este TPI permitió integrar y consolidar los contenidos principales de Programación 1 en un sistema funcional real:

- Las listas de diccionarios demostraron ser la estructura de datos más adecuada para este dominio: flexibles, legibles y fácilmente extensibles. Si en el futuro se necesitara agregar un campo (por ejemplo, idioma oficial), solo requeriría modificar la lectura del CSV y las funciones de alta, sin cambiar la lógica de filtrado ni ordenamiento.
- La modularización con funciones de responsabilidad única facilitó el desarrollo incremental y el debugging: fue posible probar cada función de forma aislada antes de integrarla al menú. El principio de separar la capa de persistencia (CSV) de la capa de lógica de negocio (filtros, estadísticas) mejoró significativamente la mantenibilidad.
- El manejo de archivos sin importar módulos externos (solo `open()` nativo) permitió comprender los mecanismos internos de lectura y escritura: qué significa un archivo de texto y cómo separar campos con `split()`.
- La validación de entradas y el manejo de excepciones con try/except resultaron esenciales para una aplicación de consola robusta: sin ellos, cualquier entrada inesperada del usuario terminaría el programa abruptamente. La experiencia reforzó el principio de nunca confiar en la entrada del usuario.

En conjunto, el proyecto evidenció que estructuras de datos simples, bien aplicadas, son suficientes para resolver problemas reales de gestión de información. La complejidad no está en las herramientas sino en su uso disciplinado.

4. Bibliografía y Webgrafía

1. Python Software Foundation. (s.f.). Documentación oficial de Python 3. Recuperado de <https://docs.python.org/3/>
2. Python Software Foundation. (s.f.). Built-in Functions — open(). Python 3 Documentation. <https://docs.python.org/3/library/functions.html#open>
3. Python Software Foundation. (s.f.). Data Structures — More on Lists and Dictionaries. Python 3 Tutorial. <https://docs.python.org/3/tutorial/datastructures.html>
4. Real Python. (s.f.). Sorting Data in Python With the sorted() Function. <https://realpython.com/python-sort/>
5. Real Python. (s.f.). Reading and Writing CSV Files in Python. <https://realpython.com/python-csv/>
6. Universidad Tecnológica Nacional — TUPAD. (2025). Material de cursada Programación 1: Unidades sobre listas, diccionarios, funciones y manejo de archivos. Plataforma educativa interna.

5. Enlaces del Proyecto

Repositorio GitHub:

<https://github.com/maraven/TPI-Programacion1.git>

Video demostrativo (YouTube):

(Agregar URL del video una vez publicado con permisos públicos — duración entre 10 y 15 minutos)

Documento PDF (informe académico):

(Agregar URL directa al PDF en el repositorio una vez subido)